第4讲　数据存储的基本形式
数据地址类型转换





国产存储芯片厂商

厂商	赛道	代表产品	市场地位
长鑫存储	DRAM	DDR5、LPDDR5X	全球 8%，国内第一
长江存储	3D NAND	294 层 NAND、致态 SSD	全球 13%，国内第一
兆易创新	NOR/DRAM	SPI NOR、利基 DRAM	NOR 全球第二
澜起科技	内存接口	DDR5 接口芯片	全球 75% 市占
江波龙	存储模组	Lexar SSD、内存条	国内模组龙头
佰维存储	SSD / 嵌入式	工业级 SSD	自研主控 + 颗粒

主要内容

- 数据存储的基本形式
- 数据地址的类型
- 地址类型的转换

- 强化内存中的内容是字节流概念
- 相同的内容 有不同的解读结果
- 解读依赖具体的机器指令或者解读场景
- 指令中含有从某地址开始读取数据长度的信息
- 数据长度 与 地址类型对应

数据存储的基本形式

内存条：用来存放程序和数据的装置

C 语 言：　char、short、int、float、double

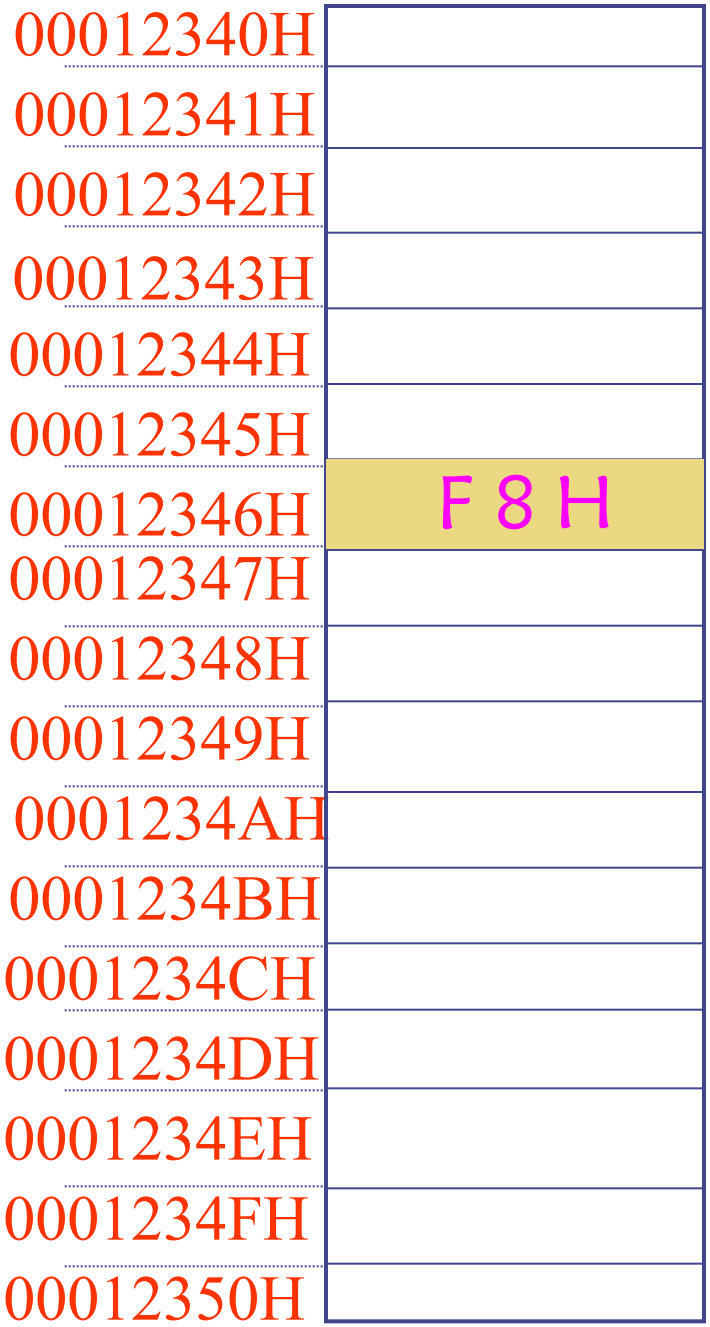
对应长度：　1个、 2个、 4个、 4个、 8个字节

	字节	字	双字	双字	四字
汇编语言：	byte	word	dword	dword	qword

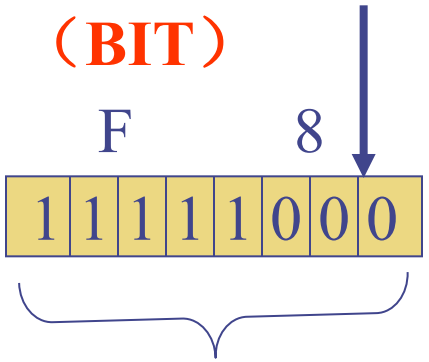
字节是最小的寻址单位

每一个字节都有一个地址

物理地址 (Physical Address, PA) 是唯一的



主存的基本存储单位是位（BIT）



8个位组成一个字节 BYTE

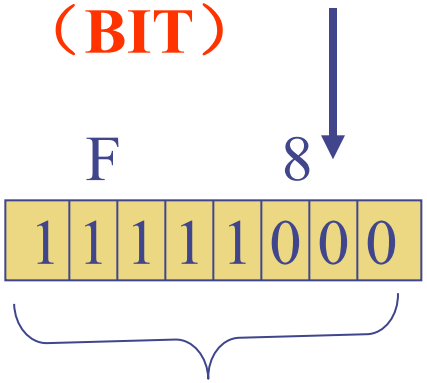
Q: 1M字节内存，地址编码需要多少二进制位？

Q:32位地址对应的内存大小可达到多大？

字节是最小的寻址单位

00012340H	
00012341H	
00012342H	
00012343H	
00012344H	
00012345H	
00012346H	F8H
00012347H	04H
00012348H	56H
00012349H	12H
0001234AH	
0001234BH	
0001234CH	
0001234DH	
0001234EH	
0001234FH	
00012350H	

主存的基本存储单位是位（BIT）



8个位组成一个字节 **BYTE**

字地址是这2个字节中低字节的地址

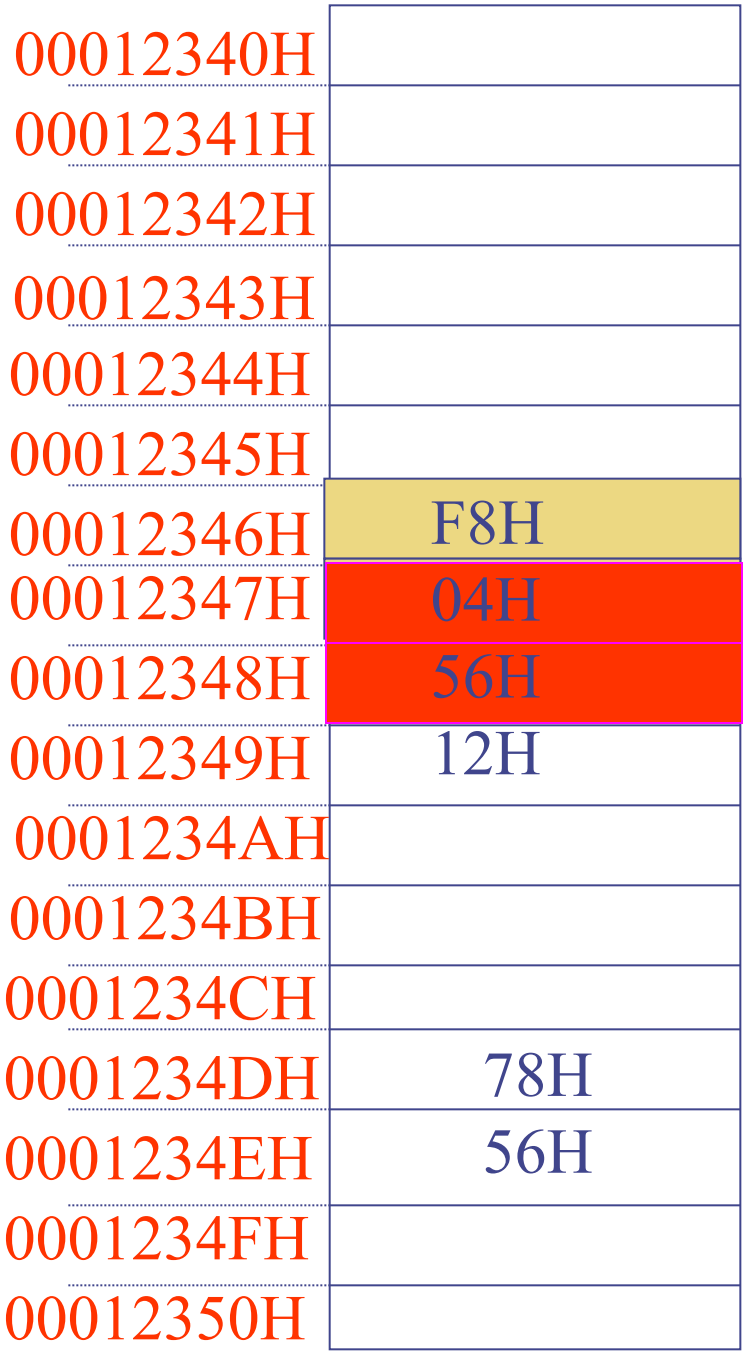
两个相邻的字节组成一个字 **WORD**

Q1:两个黄色的字节组成的字的地址是多少？字中的内容又是多少？

PA:00012346H
DATA: 04F8H

Q2:红色的呢？

PA:00012347H
DATA: 5604H



字数据的存放形式:

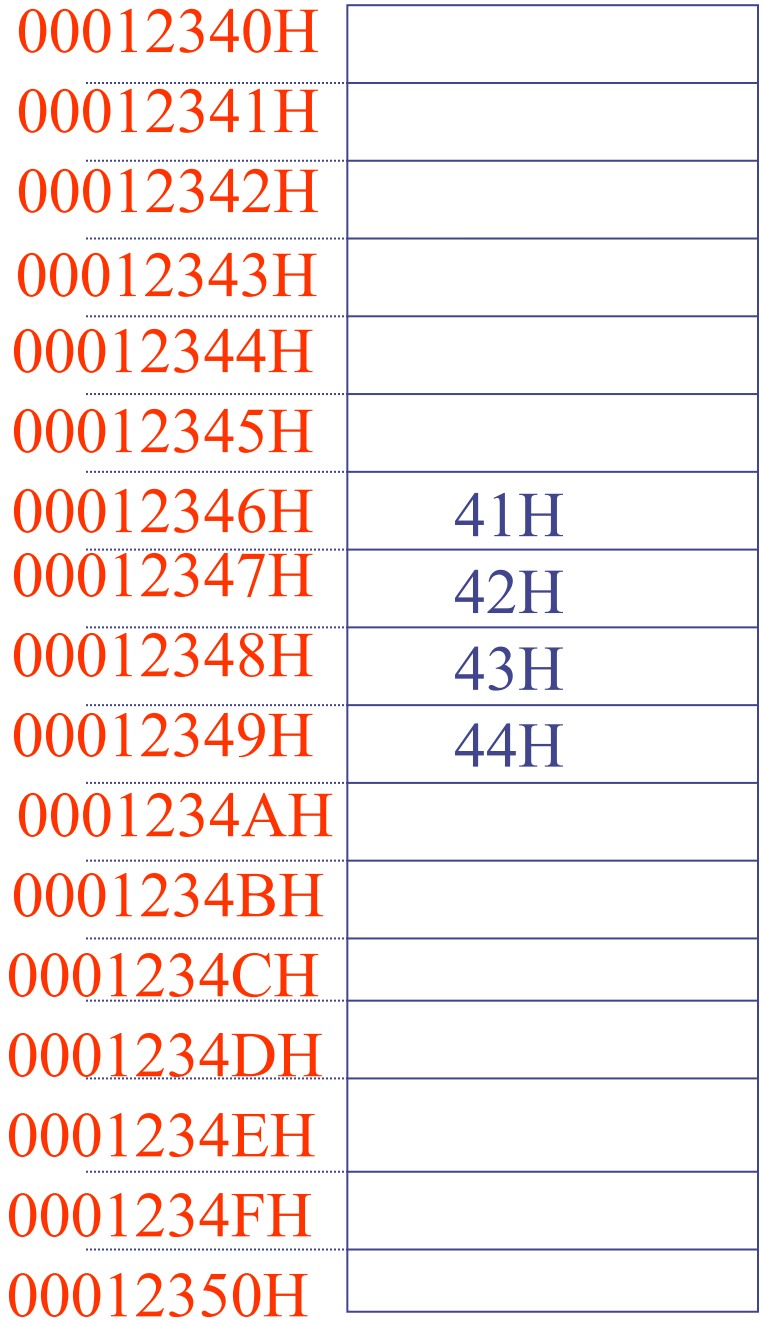
低8位在低字节；高8位在相邻的高字节中。

Q3:将5678H存放到地址为1234D的字单元中。

地址为12346H
的双字是：
44434241H
即(00012346H) =
44434241H

地址为12346H
的字是：
4241H
即(00012346H) =
4241H

地址为12346H
的字节是：
41H
即(00012346H) =
41H



双字
四个连续的字节组成

其地址为四个字节中的最低字节的地址。

数据存储的基本形式

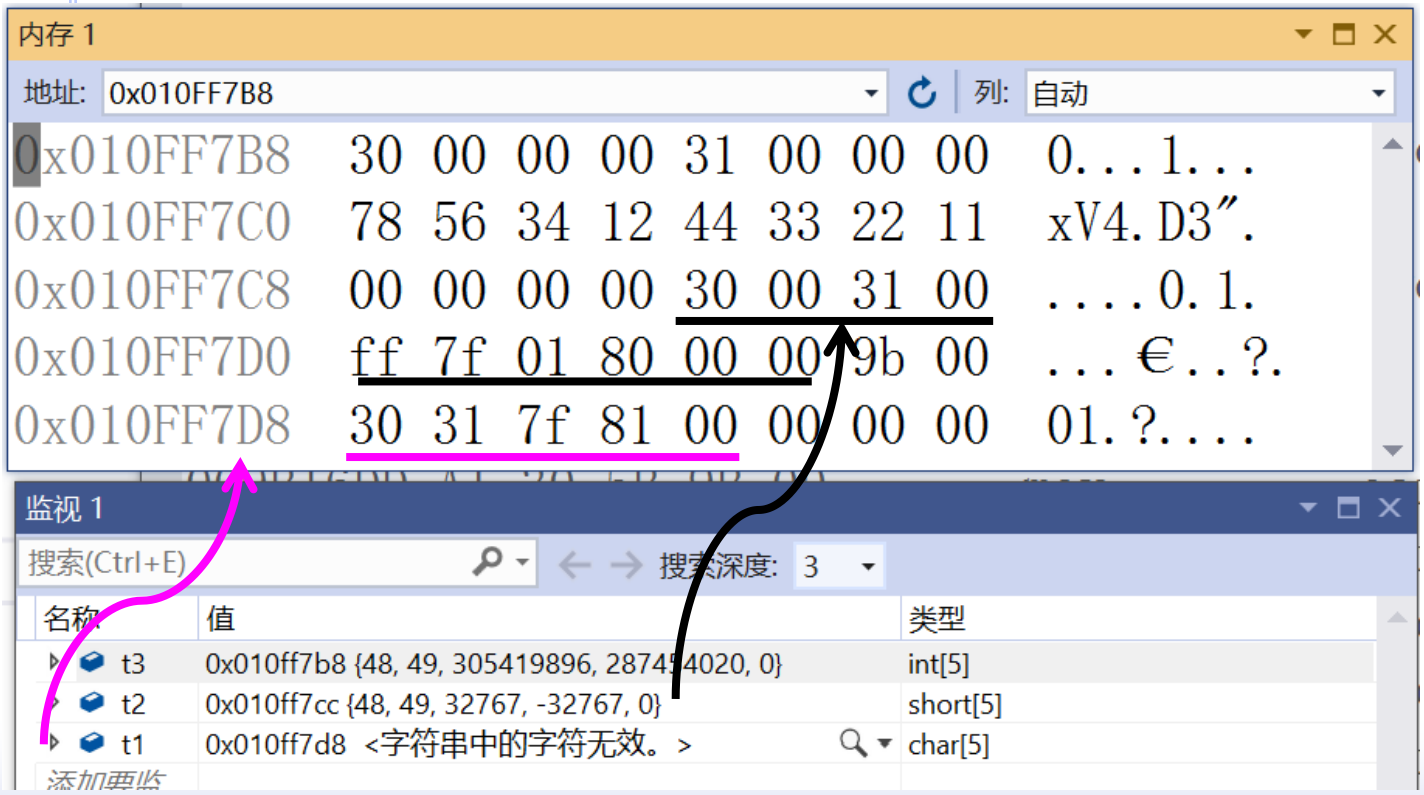
数据存储方法有两种

- 小端存储（Little Endian）
- 大端存储（Big Endian）

- Intel x86 系列采用小端存储方式
- 在小端存储方式中，最低地址字节中存放数据的最低字节，最高地址字节中存放数据的最高字节。按照数据由低字节到高字节的顺序依次存放在从低地址到高地址的单元中。
- 大端存储方式与小端存储方式相反。

数据存储的基本形式


```
char  t1[5] = { 48,49,127,129,0 };
short t2[5] = { 48,49,32767,  32769,0 };
int   t3[5] = { 48,49,0x12345678,0x11223344,0 };
```



要点：

(1) 变量对应的地址，确定存储位置

(2) 数据的表示

(3)数据存放规则

每个元素所占的字节数；

小端存放

数据存储的基本形式


```
char  t1[5] = { 48,49,127,129,0 };
short t2[5] = { 48,49,32767,  32769,0 };
int   t3[5] = { 48,49,0x12345678,0x11223344,0 };
int   *pt1 = t1; // 警告: 从“char *”到“int *”的类型不兼容
int   q = *pt1;
```

q=?

int * pt1 = (int *)t1;

内存 1

地址: 0x010FF7D8　　列: 自动

0x010FF7D8	30	31	7f	81	00	00	00	00	01.?....
0x010FF7E0	ab	23	05	2d	04	f8	0f	01	?#.-.?..

监视 1

搜索(Ctrl+E)　　搜索深度: 3

名称	值	类型
pt1	0x010ff7d8 {-2122370768}	int *
q	-2122370768	int
t1	0x010ff7d8 <字符串中的字符无效。>	char[5]
q,x	0x817f3130	int
&pt1	0x010ff7b4 {0x010ff7d8 {-2122370768}}	int * *
添加要监		

数据地址的类型及转换

给定一个地址后，可以根据该地址取一个字节、一个字、一个双字……　取多少字节的数据，取决于地址类型。

以 … 为字节地址

以 … 为字地址

以 … 为双字地址

C 语言程序中，可以灵活的进行地址类型转换，从而访问同一位置的不同长度的数据。

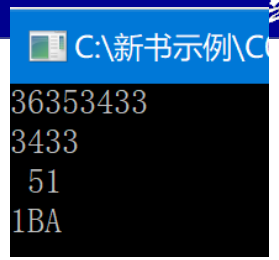
数据地址的类型及转换

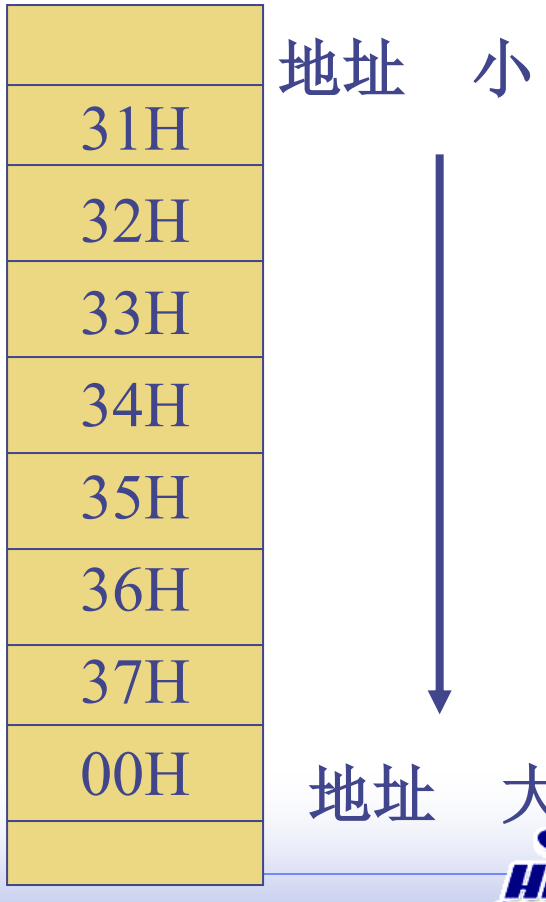
華中科技大學

程序运行结果是什么？为什么？

```
char   s[10];
strcpy(s, ”1234567”);
printf(“%x \n”,  *(long *)(s+2));
printf(“%x \n”,  *(short *)(s+2));
printf(“%d \n”,  *(char *)(s+2));
*(int *)(s+1)=16706;
printf(“%s \n”,s);
```

关键词：地址类型转换



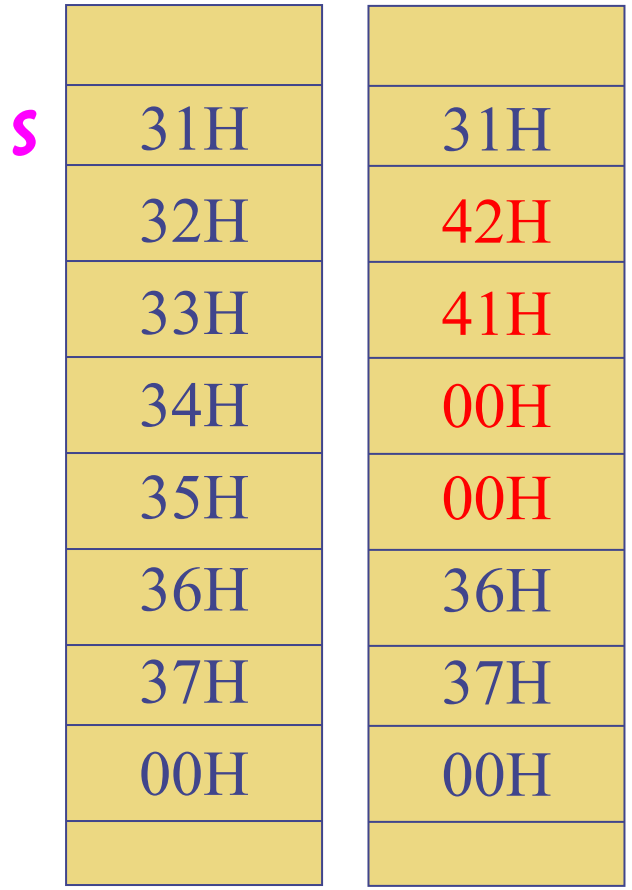


数据地址的类型及转换


```
*(int *)(s+1)=16706;
printf("%s \n",s);
```

16706 = 0x4142
= 0x 00 00 41 42

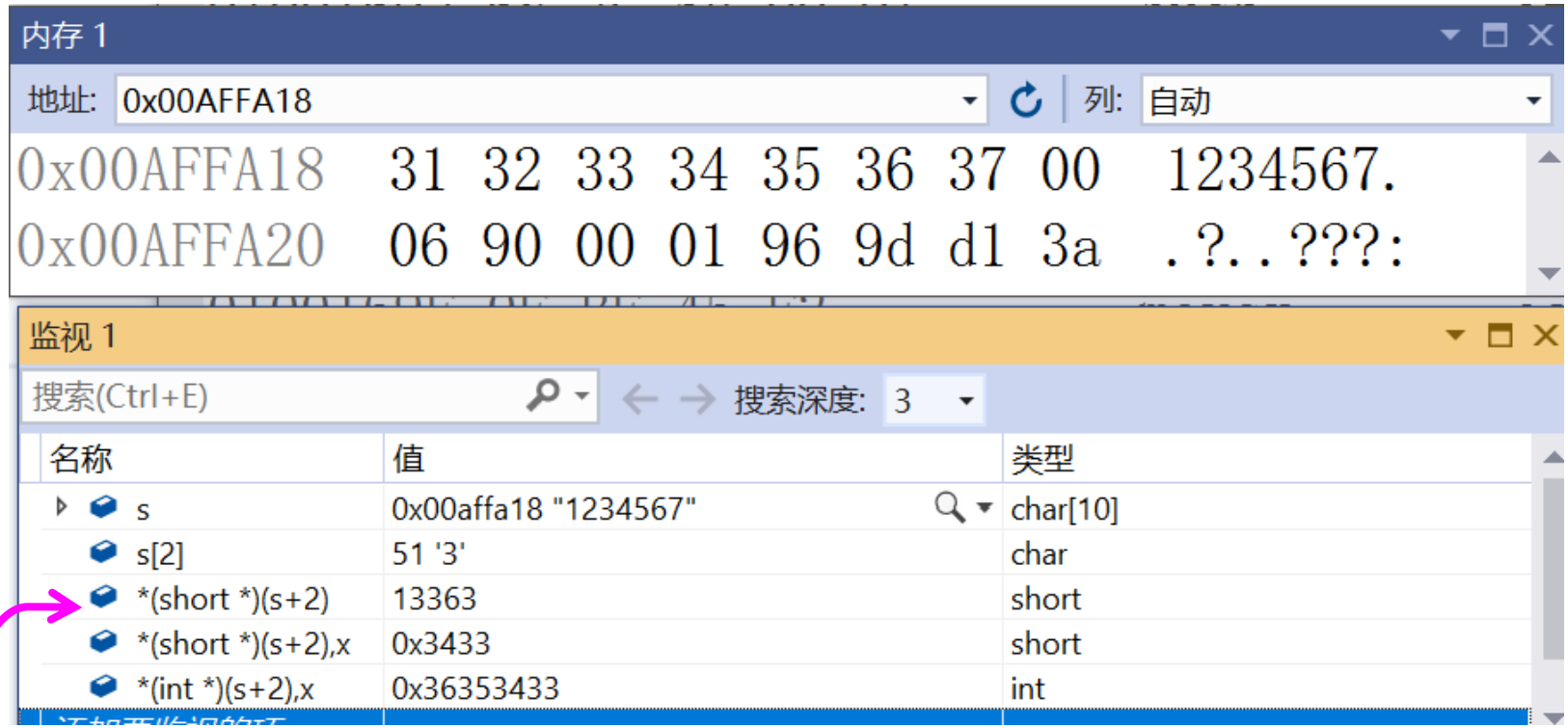
显示：1BA
‘A’的ASCII 为 0x41



数据地址的类型及转换


```
char   s[10];
strcpy(s, ”1234567”);
```

地址类型转换

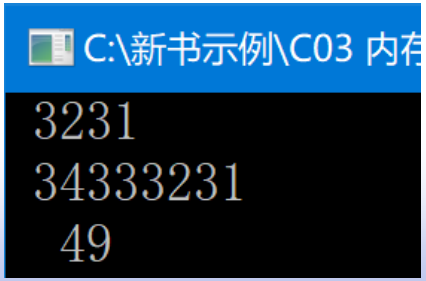


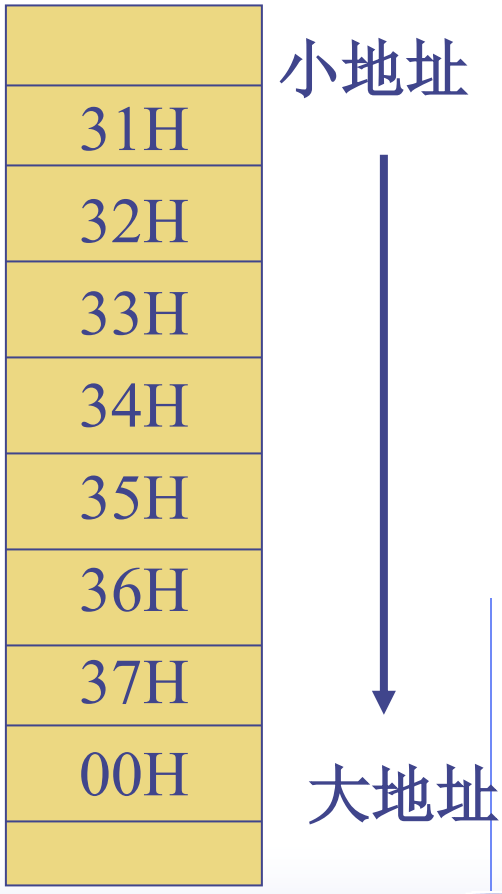
在调试窗口，观察地址类型转换后取数的结果

数据地址的类型及转换

地址类型转换

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
union test {
    char s[10];
    char c;
    short x;
    int  y;
} temp;
int main()
{   strcpy(temp.s, "1234567");
    printf(" % x \n", temp.x);
    printf(" % x \n", temp.y);
    printf(" % d \n", temp.c);
    return 0;
}
```





数据地址的类型及转换

地址类型转换

```
union test {
    char s[10];
    char c;
    short x;
    int   y;
}temp;
strcpy(temp.s, "1234567");
```

监视 1

搜索(Ctrl+E)　搜索深度: 3

名称	值	类型
&temp.x	0x0092a138 {union_type.exe!test temp} {12849}	short *
&temp.y	0x0092a138 {union_type.exe!test temp} {875770417}	int *
&temp.c	0x0092a138 "1234567"	char *
temp.s	0x0092a138 "1234567"	char[10]

数据地址的类型的表示

在指令的机器码中，有操作数类型的编码

Intel 格式的汇编语言中：
在操作数地址前用 *** ptr 来表明操作数类型
byte, word, dword, qword 分别对应 1、 2、 4、 8 字节

```
   14:      char x1 = 0;
mov          byte ptr [ebp-5],0
   15:      short x2 = 0;
xor          eax,eax
mov          word ptr [ebp-14h],ax
   16:      int   x4 = 0;
mov          dword ptr [ebp-20h],0
   17:      long  long x8 = 0;
xorps        xmm0,xmm0
movlpd       qword ptr [ebp-30h],xmm0
```

数据地址的类型的表示

在指令的机器码中，有操作数类型的编码

Intel 格式的汇编语言中：
在操作数地址前用 *** ptr 来表明操作数类型

AT&T 格式的汇编语言中,在操作符中有后缀：
b，w，l，q 分别对应 1、 2、 4、 8 字节

```
5           char  x1=0;
   0x0000555555555151 <+8>:     movb   $0x0,-0xf(%rbp)
6          short  x2=0;
   0x0000555555555155 <+12>:    movw   $0x0,-0xe(%rbp)
7          int    x4=0;
   0x000055555555515b <+18>:    movl   $0x0,-0xc(%rbp)
8          long long x8=0;
   0x0000555555555162 <+25>:    movq   $0x0,-0x8(%rbp)
```

结构变量的空间分配


```
#include <stdio.h>
struct  struct_test {
    char    c1;
    double  d8;
    short   s2;
    int     i4;
};
int main()
{   struct_test s = { '1',10.5, 20,30 };
    printf("sizeof struct : %d\n", sizeof(s));
    printf("address %p %p %p %p \n",
           &s.c1, &s.d8 , &s.s2, &s.i4 );
    return 0;
}  //s大小等同 sizeof(struct_test);
```

内存 1

地址: 0x00B3F754

```
0x00B3F754   31 cc cc cc cc cc cc cc
0x00B3F75C   00 00 00 00 00 00 25 40
0x00B3F764   14 00 cc cc 1e 00 00 00
```

```
C:\教学\本科教学\计算机系统基础\计算机系统基础_程序\C03_数据的机内表示\数据的机内表示\Debug\struct_test.exe
sizeof struct : 24
address 00B3F754 00B3F75C 00B3F764 00B3F768
```

结构变量的空间分配

```
struct  struct_test {
    char    c1;
    double  d8;
    short   s2;
    int     i4;
};
```

```
C:\教学\本科教学\计算机系统基础\计算机系统基础_程序\C03_数据的机内表示\数据的机内表示\Debug\struct_test.exe
sizeof struct : 24
address 00B3F754 00B3F75C 00B3F764 00B3F768
```

- **字段顺序**：各字段在内存中的出现顺序与声明顺序一致（编译器理论上可重排，但大多数实际不会）。
- **字节对齐规则**
  - 结构体首地址能被其最宽基本类型成员大小整除。
  - 每个成员相对于结构体首地址的偏移量，必须是该成员大小的整数倍，不足则填充 padding。
  - 结构体总大小对齐到最宽成员大小的整数倍。
- **目的**：提高 CPU 访问内存的效率。

结构变量的空间分配


```
#include <stdio.h>
#pragma pack(1)
struct  struct_test {
    char    c1;
    double  d8;
    short   s2;
    int     i4;
};
int main()
{   struct_test s = { '1',10.5, 20,30 };
    printf("sizeof struct : %d\n", sizeof(s));
    printf("address %p %p %p %p \n",
           &s.c1, &s.d8 , &s.s2, &s.i4 );
    return 0;
}  //s大小等同 sizeof(struct_test);
```

不同的编译开关
分配的结果不同！

```
sizeof struct : 15
address 008FF758 008FF759 008FF761 008FF763
```

以不同形式观察内存




```
0x0133FAEC   31 cc cc cc cc cc cc cc
0x0133FAF4   00 00 00 00 00 00 25 40
0x0133FAFC   14 00 cc cc 1e 00 00 00

0x0133FAEC   cc31 cccc cccc cccc
0x0133FAF4   0000 0000 0000 4025
0x0133FAFC   0014 cccc 001e 0000

0x0133FAEC   cccccc31 cccccccc
0x0133FAF4   00000000 40250000
0x0133FAFC   cccc0014 0000001e
```

数据地址的类型及转换

练习：体会地址类型转换的应用

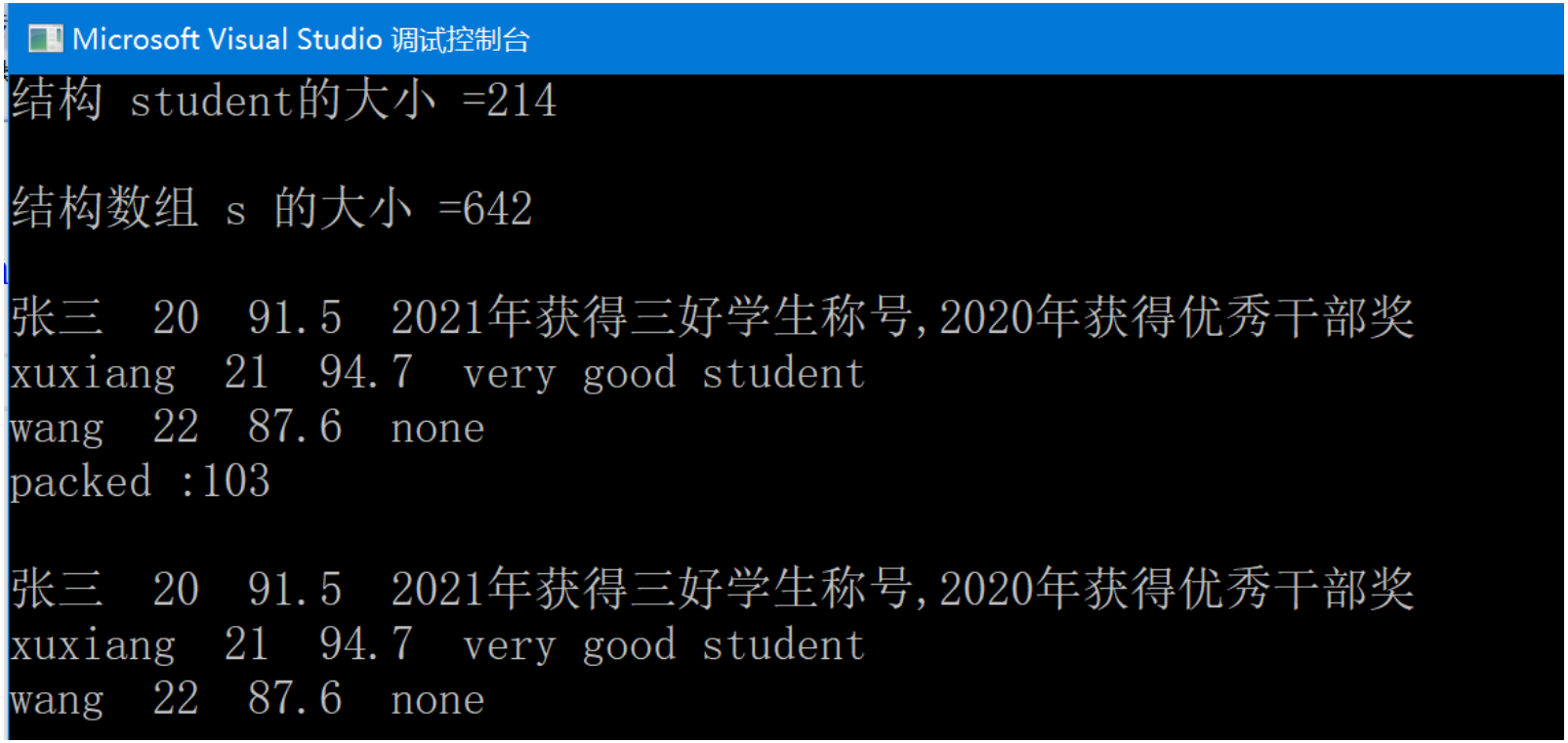
定义了 结构 student ， 以及结构数组变量s[3];

```
struct student {
    char  name[8];
    short age;
    float  score;
    char  remark[200];
};
student  s[3];
student  new_s[3];
```

编写程序，将 s[3] 中的信息紧凑存放到 一个字符数组 message 中，然后从 message 转换到结构数组 new_s[3]中。

数据地址的类型及转换

练习



要求 s[0].name 为自己的姓名;
可以在给的程序中 补充

```
void pack_student(student* s)
void restore_student(student* s)
```

小结

- 整型数据的存放规律：长度与类型有关、小端存放；
- 内存中存放的数据串是一个字节流；
- 从同一个地址开始，用不同类型来解读，得到不同结果；

  [从一个地址开始的 char、short、int、float、double]

  [unsigned char、unsigned short、unsigned int]
- 每个变量都有一个地址；
- 变量地址的表达形式与编译器有关；
- 不同的编译开关可生成不同的地址表达形式。